

torch.nn.utils.memory_format.convert_c

[https://pytorch.org/docs/stable/generated/torch.nn.utils.memory_format.convert](https://pytorch.org/docs/stable/generated/torch.nn.utils.memory_format.convert_conv2d_weight_memory_format.html)

Description:

Convert `memory_format` of `nn.Conv2d.weight` to `memory_format`. The conversion recursively applies to nested `nn.Module`, including `module`. Note that it only changes the `memory_format`, but not the semantics of each dimensions. This function is used to facilitate the computation to adopt NHWC kernels, which provides considerable speed up for fp16 data on CUDA devices with compute capability ≥ 7.0 . Calling `model.to(memory_format=torch.channels_last)` is more aggressive than the utility function `convert_conv2d_weight_memory_format`. Any layer with 4d weight will be affected by `model.to`, which does not necessarily benefit from conversion to specified `memory_format`. One place we are confident in is that NHWC(channels_last) conversion for convolution in cuDNN, as it is beneficial to run convolution in NHWC, even in cases where we have to apply permutation to input tensors. Hence our strategy here is to convert only the weight of convolution to `channels_last`. This ensures that; 1. Fast convolution kernels will be used, the benefit of which could outweigh overhead of permutation (if input is not in the same format). 2. No unnecessary permutations are applied on layers that do not benefit from m

Function Signature

```
torch.nn.utils.memory_format.convert_conv2d_weight_memory_format(module, memory_format)[source]#
```

Parameters

Returns

Return type

Returns:

_M

Code Examples

```
>>> input = torch.randint(
...     1, 10, (2, 8, 4, 4), dtype=torch.float16, device="cuda"
... )
>>> model = nn.Sequential(
>>>     nn.Conv2d(8, 4, 3)).cuda().half()
>>> # This is identical to:
>>> # nn.utils.convert_conv2d_weight_memory_format(model,
torch.channels_last)
>>> model = nn.utils.convert_conv2d_weight_memory_format(
...     model, torch.channels_last
... )
>>> out = model(input)
```

Notes & Warnings

NOTE

Note Calling `model.to(memory_format=torch.channels_last)` is more aggressive than the utility function `convert_conv2d_weight_memory_format`. Any layer with 4d weight will be affected by `model.to`, which does not necessarily benefit from conversion to specified `memory_format`. One place we are confident in is that NHWC(channels_last) conversion for convolution in cuDNN, as it is beneficial to run convolution in NHWC, even in cases where we have to apply permutation to input tensors. Hence our strategy here is to convert only the weight of convolution to channels_last. This ensures that; 1. Fast convolution kernels will be used, the benefit of which could outweigh overhead of permutation (if input is not in the same format). 2. No unnecessary permutations are applied on layers that do not benefi

Detailed Documentation

Documentation

Convert `memory_format` of `nn.Conv2d.weight` to `memory_format`.

The conversion recursively applies to nested `nn.Module`, including module. Note that it only changes the `memory_format`, but not the semantics of each dimensions. This function is used to facilitate the computation to adopt NHWC kernels, which provides considerable speed up for fp16 data on CUDA devices with compute capability ≥ 7.0

Calling `model.to(memory_format=torch.channels_last)` is more aggressive than the utility function `convert_conv2d_weight_memory_format`. Any layer with 4d weight will be affected by `model.to`, which does not necessarily benefit from conversion to specified `memory_format`. One place we are confident in is that NHWC(channels_last) conversion for convolution in cuDNN, as it is beneficial to run convolution in NHWC, even in cases where we have to apply permutation to input tensors.

Hence our strategy here is to convert only the weight of convolution to `channels_last`. This ensures that; 1. Fast convolution kernels will be used, the benefit of which could outweigh overhead of permutation (if input is not in the same format). 2. No unnecessary permutations are applied on layers that do not benefit from `memory_format` conversion.

The optimal case is that, layers between convolution layers are channels last compatible. Input tensor would be permuted to channels last when it encounters the first convolution layer and stay in that memory format. Hence following convolutions will not need to permute its input tensor.

In case where a channels last incompatible layer is between convolution layers, we need to permute the input tensor back to contiguous format for that layer. The input tensor will go through the remaining layers in contiguous format and be permuted to channels last when it encounters another convolution layer. There's no point in propagating that permutation to an earlier layer, as most layers are quite agnostic to `memory_format`.

This claim might change when PyTorch supports fusion of permutation, as there might have been a better spot to fuse the permutation other than immediately before a convolution.

module (`nn.Module`) – `nn.Conv2d` & `nn.ConvTranspose2d` or container `nn.Module`

memory_format (memory_format) – user specified memory_format, e.g. torch.channels_last or torch.contiguous_format

The original module with updated nn.Conv2d